

Relazione Tecnica: Simulazione di una Rete a Due Nodi con Feedback e Priorità

Leonardo Vincenzi 0001056530

Simulazione di Sistemi
2024/2025

1 Introduzione e Abstract

Il presente progetto si pone l'obiettivo di analizzare le prestazioni di un sistema a coda distribuito, modellando una rete a due nodi con meccanismi di feedback ritardato. Il modello di riferimento è una variante specifica del sistema descritto nell'articolo "*Response times in a two-node queueing network with feedback*" di R.D. van der Mei et al.

A differenza del modello originale, che prevede un nodo *Processor Sharing* (PS) e un nodo FCFS generico, la variante oggetto di studio (Progetto n. 8) introduce delle varianti legate alla gestione di due distinte tipologie di utenti, U_1 e U_2 . Tali utenti sono caratterizzati da diverse priorità di servizio e da instradamenti specifici all'interno della rete. La simulazione, realizzata mediante il framework OMNeT++, mira a stimare misure di prestazione critiche quali la lunghezza media delle code, i tempi di soggiorno nel sistema e la percentuale di utenti che superano una determinata soglia di permanenza.

2 Concetti generali sul modello

Il sistema è modellato come una rete di code aperta costituita da due nodi di servizio principali in serie, con un ciclo di feedback.

1. **Ingresso e Tipologie:** Gli utenti arrivano al sistema da un'unica sorgente esterna. Al momento della creazione, a ciascun utente viene assegnata una classe: tipo U_1 (alta priorità) con probabilità q , o tipo U_2 (bassa priorità) con probabilità $1 - q$.
2. **Nodo 1 (Servente Prioritario):** Il primo nodo è costituito da un singolo servente che adotta una disciplina a priorità: gli utenti U_1 vengono serviti sempre prima degli utenti U_2 . I tempi di servizio seguono una distribuzione uniforme con parametri differenziati per le due classi.
3. **Routing e Feedback:** Al termine del servizio presso il Nodo 1, l'utente può lasciare definitivamente il sistema con probabilità $(1 - p)$, oppure richiedere un ulteriore servizio al Nodo 2 con probabilità p (feedback).
4. **Nodo 2 (Serventi Paralleli):** Il secondo nodo è composto da due serventi dedicati: $SS1$ serve esclusivamente gli utenti U_1 , mentre $SS2$ serve solo gli utenti U_2 . In questo nodo i tempi di servizio sono distribuiti esponenzialmente. Gli utenti in uscita dal Nodo 2 rientrano sempre nel Nodo 1 (feedback deterministico).

3 Descrizione delle componenti e del codice

Il modello è stato implementato definendo moduli semplici C++ che estendono la classe `cSimpleModule` di OMNeT++. Di seguito si analizza il funzionamento logico e implementativo delle componenti.

3.1 Componente Source

Il modulo **Source** è responsabile della generazione del traffico entrante nella rete. I tempi di interarrivo degli utenti seguono una distribuzione esponenziale negativa con media m .

Nel codice, la funzione `handleMessage` gestisce la creazione del messaggio `Job`. Viene utilizzato un generatore di numeri casuali uniforme per determinare la tipologia dell'utente (U_1 o U_2) confrontando il valore estratto con il parametro q (`userTypeProbability`).

Snippet Source.cc:

```
1 void Source::handleMessage(cMessage *msg) {
2     // Genera un nuovo utente
3     Job *job = new Job("job");
4     job->setCreationTime(simTime());
5
6     // Assegna il tipo di utente in base alla probabilita' q
7     if (uniform(0, 1) < par("userTypeProbability").
doubleValue()) {
8         job->setType(1); // Utente U1
9     } else {
10        job->setType(2); // Utente U2
11    }
12
13    send(job, "out");
14
15    // Schedula il prossimo arrivo usando una distribuzione
    ESPONENZIALE
16    scheduleAt(simTime() + exponential(par("interArrivalTime"
    ).doubleValue()), sendMessageEvent);
17 }
```

3.2 Componente Node1

Il **Node1** rappresenta il cuore della logica prioritaria del sistema. Gestisce due code interne, `queueU1` e `queueU2`. Quando il server si libera, la funzione `startNextJob` verifica prima la presenza di utenti nella coda prioritaria (U_1); solo se questa è vuota, procede a servire un utente di tipo U_2 .

Inoltre, il nodo gestisce il routing probabilistico post-servizio. Il tempo di servizio è calcolato tramite distribuzione uniforme su intervalli $[a, b]$ o $[c, d]$ a seconda del tipo di utente.

Snippet Node1.cc:

```
1 void Node1::startNextJob() {
2     // Disciplina prioritaria: serve prima gli utenti U1
3     if (!queueU1.isEmpty()) {
4         jobInService = (Job*)queueU1.pop();
5         emit(qLenSignalU1, queueU1.getLength());
6     } else if (!queueU2.isEmpty()) {
7         jobInService = (Job*)queueU2.pop();
8         emit(qLenSignalU2, queueU2.getLength());
9     } else {
10        return; // Niente da servire
11    }
12
13    // ... calcolo serviceTime ...
```

```

14 // Tempo di servizio uniforme
15 if (jobInService->getType() == 1) {
16     serviceTime = uniform(par("serviceTimeU1_a").
doubleValue(), par("serviceTimeU1_b").doubleValue());
17 } else {
18     serviceTime = uniform(par("serviceTimeU2_c").
doubleValue(), par("serviceTimeU2_d").doubleValue());
19 }
20 scheduleAt(simTime() + serviceTime, endServiceEvent);
21 }

```

3.3 Componente Node2

Il **Node2** agisce strutturalmente come un contenitore per i due serventi dedicati. Sebbene nel file `.ned` appaia come un singolo modulo, nel codice C++ gestisce logicamente due linee di servizio parallele e indipendenti. All'arrivo di un messaggio, il metodo `handleMessage` smista immediatamente il job nella coda corretta (`queueSS1` o `queueSS2`) in base al tipo, garantendo l'isolamento tra i flussi U_1 e U_2 .

Snippet Node2.cc:

```
1 void Node2::handleMessage(cMessage *msg) {
2     // ... gestione eventi fine servizio ...
3
4     Job *job = check_and_cast<Job *>(msg);
5     // SS1 serve solo U1, SS2 serve solo U2
6     if (job->getType() == 1) {
7         queueSS1.insert(job);
8         emit(qLenSignalSS1, queueSS1.getLength());
9         if (!isBusySS1) startNextJobSS1();
10    } else {
11        queueSS2.insert(job);
12        emit(qLenSignalSS2, queueSS2.getLength());
13        if (!isBusySS2) startNextJobSS2();
14    }
15 }
```

3.4 SS1 Servente 1

Il servente **SS1** è implementato logicamente all'interno della classe **Node2**. Esso preleva job esclusivamente dalla **queueSS1**. La caratteristica fondamentale di questo componente è la distribuzione del tempo di servizio, che è di tipo esponenziale con media $m(SS1)$. Poiché il servente **SS1** opera indipendentemente da **SS2**, utilizza una variabile di stato dedicata **isBusySS1** e un evento di auto-messaggio **endServiceSS1Event**.

Snippet Node2.cc:

```
1 void Node2::startNextJobSS1() {
2     if (queueSS1.isEmpty()) return;
3     jobInServiceSS1 = (Job*)queueSS1.pop();
4
5     isBusySS1 = true;
6     // Tempo di servizio esponenziale per SS1
7     simtime_t serviceTime = exponential(par("serviceTimeSS1")
8     .doubleValue());
9     scheduleAt(simTime() + serviceTime, endServiceSS1Event);
10 }
```

3.5 SS2 Servente 2

Analogamente a **SS1**, il servente **SS2** gestisce il flusso degli utenti di tipo U_2 prelevandoli dalla **queueSS2**. Anche in questo caso, il tempo di servizio segue una distribuzione esponenziale, ma con un parametro di media differente, $m(SS2)$. La sua logica di esecuzione è parallela a quella di **SS1**, permettendo al Nodo 2 di servire contemporaneamente un utente U_1 e un utente U_2 .

Snippet Node2.cc:

```
1 void Node2::startNextJobSS2() {
2     if (queueSS2.isEmpty()) return;
3     jobInServiceSS2 = (Job*)queueSS2.pop();
4
5     isBusySS2 = true;
6     // Tempo di servizio esponenziale per SS2
7     simtime_t serviceTime = exponential(par("serviceTimeSS2")
8     .doubleValue());
9     scheduleAt(simTime() + serviceTime, endServiceSS2Event);
10 }
```

3.6 Componente Sink

Il modulo **Sink** rappresenta il punto di uscita del sistema e il centro di raccolta delle statistiche. La sua funzione principale è ricevere i job che hanno completato il loro ciclo di vita (sia dopo il passaggio nel Nodo 1 che, eventualmente, dopo diversi cicli di feedback), calcolare le metriche di interesse e distruggere l'oggetto per liberare memoria.

Al ricevimento di un messaggio, il Sink calcola il *tempo di soggiorno* (sojourn time) sottraendo il tempo di creazione del job (`creationTime`) dal tempo di simulazione corrente. Questo valore viene poi emesso tramite segnali specifici per gli utenti U_1 e U_2 , permettendo la raccolta di statistiche come media, massimo, minimo e distribuzioni.

Snippet Sink.cc:

```
1 void Sink::handleMessage(cMessage *msg) {
2     Job *job = check_and_cast<Job *>(msg);
3
4     // Calcolo del tempo di permanenza nel sistema
5     simtime_t sojournTime = simTime() - job->getCreationTime
6     ();
7
8     // Registrazione delle statistiche per tipo di utente
9     if (job->getType() == 1) {
10         emit(sojournTimeSignalU1, sojournTime);
11     } else {
12         emit(sojournTimeSignalU2, sojournTime);
13     }
14
15     // Eliminazione del job dal sistema
16     delete job;
17 }
```

3.7 Feedback

Il meccanismo di **Feedback** è una logica distribuita tra il Nodo 1 e la connessione con il Nodo 2. Non esiste un componente "Feedback" isolato; piuttosto, il routing è determinato probabilisticamente al termine del servizio nel Nodo 1.

Come specificato nelle modifiche al modello, un utente servito dal Nodo 1 può lasciare il sistema o richiedere ulteriore servizio.

- Con probabilità p , il job viene instradato verso il Nodo 2 (`toNode2`).
- Con probabilità $1 - p$, il job viene instradato verso il Sink (`toSink`).

Gli utenti che entrano nel Nodo 2 e vengono serviti da *SS1* o *SS2* vengono poi reimmessi deterministicamente nel Nodo 1 attraverso un ciclo di ritorno, trattati come nuovi arrivi ma mantenendo il loro tempo di creazione originale.

Snippet Node1.cc:

```
1 if (msg == endServiceEvent) {
2     // Decisione di routing basata su p (feedbackProbability)
3     if (uniform(0, 1) < par("feedbackProbability").
4         doubleValue()) {
5         send(jobInService, "toNode2"); // Feedback: va al
6         nodo 2
7     } else {
8         send(jobInService, "toSink"); // Uscita: lascia il
9         sistema
10    }
11    // ... reset stato servente ...
12 }
```

3.8 Job

L'entità **Job** è un messaggio personalizzato definito nel file `Job.msg`. Esso estende la classe base dei messaggi di OMNeT++ trasportando le informazioni essenziali per la logica di controllo e per le statistiche:

1. **Type:** Un intero che identifica la classe dell'utente (1 per U_1 , 2 per U_2). Questo campo è immutabile per la vita del job e determina le priorità e il routing interno al Nodo 2.
2. **CreationTime:** Il timestamp della generazione del job, necessario per calcolare il tempo totale di risposta del sistema nel Sink.

Snippet Job.msg:

```

1 message Job {
2     int type;                // 1 = U1 (Alta Priorita), 2 = U2 (
    Bassa Priorita)
3     simtime_t creationTime; // Tempo di ingresso nel sistema
4 }

```

3.9 Rete principale

La rete, definita nel file `Network.ned`, orchestra le connessioni tra i moduli descritti precedentemente. La topologia riflette la struttura a due nodi con feedback descritta nelle specifiche di progetto.

Il funzionamento del flusso di rete è il seguente:

1. La **Source** genera i job e li invia al **Node1**.
2. Il **Node1** possiede un vettore di gate di ingresso (`in[]`) per gestire due flussi distinti che convergono nello stesso buffer di attesa:
 - `in[0]`: riceve i nuovi arrivi dalla Source.
 - `in[1]`: riceve i job di ritorno (feedback) dal Node2.
3. L'uscita del Node1 si biforca verso il **Node2** (per il feedback) o verso il **Sink** (per l'uscita), realizzando il routing probabilistico.
4. L'uscita del **Node2** è collegata circolarmente all'ingresso `in[1]` del Node1, chiudendo l'anello di feedback.

Snippet della topologia `Network.ned`:

```

1 connections:
2     source.out    --> node1.in[0];    // Nuovi arrivi
3     node2.out     --> node1.in[1];    // Ritorno dal feedback
4
5     node1.toNode2 --> node2.in;       // Instradamento verso
    il feedback
6     node1.toSink  --> sink.in;        // Instradamento verso l
    'uscita

```


4 Parametri di Simulazione

I parametri che governano il comportamento stocastico del sistema sono stati definiti in conformità con le specifiche del Progetto n. 8. Di seguito si riporta la tabella completa dei parametri utilizzati nelle diverse configurazioni di simulazione, con la relativa spiegazione.

Parametro	Descrizione	Valori / Distribuzione
m	Media dei tempi di interarrivo alla sorgente (distribuzione esponenziale). Un valore più basso indica un traffico in ingresso più intenso.	$\{4.0, 5.0, 6.0\}$ s
q	Probabilità che un nuovo utente generato sia di tipo U_1 (alta priorità). Il complemento $(1 - q)$ è la probabilità per U_2 .	$\{0.4, 0.6, 0.8\}$
p	Probabilità di feedback. Indica la probabilità con cui un utente, dopo il Nodo 1, richiede servizio al Nodo 2 invece di uscire.	$\{0.6, 0.8, 0.9\}$
$[a, b]$	Estremi dell'intervallo per la distribuzione uniforme del tempo di servizio al Nodo 1 per utenti U_1 .	$[1.4, 2.6]$ s $[2.8, 3.2]$ s
$[c, d]$	Estremi dell'intervallo per la distribuzione uniforme del tempo di servizio al Nodo 1 per utenti U_2 .	$[1.0, 4.0]$ s $[1.8, 4.2]$ s
$m(SS1)$	Media del tempo di servizio (esponenziale) del servente dedicato agli utenti U_1 nel Nodo 2.	$\{2.4, 3.0, 3.5\}$ s
$m(SS2)$	Media del tempo di servizio (esponenziale) del servente dedicato agli utenti U_2 nel Nodo 2.	$\{2.0, 3.0, 4.0\}$ s

Tabella 1: Tabella dei parametri di simulazione

5 Analisi del Transiente

L'analisi dei sistemi a coda tramite simulazione richiede una distinzione netta tra la fase transitoria iniziale e il regime stazionario (*steady-state*). L'obiettivo di questa fase è determinare la lunghezza del **warmup-period**, ovvero l'intervallo di tempo iniziale durante il quale i dati raccolti sono influenzati dalle condizioni di partenza (come il sistema vuoto) e devono essere scartati per non introdurre bias nelle stime statistiche.

5.1 Metodologia e Run Pilota

Per valutare la stabilità del sistema, sono state eseguite delle simulazioni pilota (*Run Pilota*) configurando lo scenario in due modalità estreme:

- **Worst Case:** Configurazione ad alto carico. In questo scenario il sistema si è rivelato instabile con un transiente in continua crescita, rendendolo inadatto all'identificazione di un punto di equilibrio.
- **Best Case (Stable):** Configurazione a basso carico. Questa configurazione ha permesso di osservare il raggiungimento di un regime stazionario e di identificare il punto temporale in cui le metriche si stabilizzano.

5.2 Scelta della Metrica di Riferimento

La determinazione del termine del transiente deve basarsi sulla metrica che converge più lentamente al valore stazionario (regola del "*slowest converge metric*"). Nel sistema in esame, caratterizzato da una disciplina a priorità al Nodo 1 ($U_1 > U_2$), la classe U_2 subisce le maggiori variazioni e ritardi. Confrontando le metriche ottenute dalla Run Pilota:

Tabella 2: Risultati del Run Pilota (Configurazione: WarmupPilot_Stable, Rep: #0)

Module	Metric Name	Count	Mean	StdDev
node1	queueLenNode1_U1:vector	66 945	~ 0.819	~ 0.803
node1	queueLenNode1_U2:vector	99 451	~ 16.978	~ 19.441
node2	queueLenSS1:vector	40 173	~ 0.554	~ 0.573
node2	queueLenSS2:vector	59 549	~ 0.561	~ 0.586
sink	sojournTimeU1:vector	13 386	~ 14.433 s	~ 14.593 s
sink	sojournTimeU2:vector	19 949	~ 175.091 s	~ 271.902 s

Di conseguenza, è stato scelto il vettore `queueLenNode1_U2:vector` come indicatore di riferimento, in quanto rappresenta la componente più lenta e critica del sistema.

5.3 Tecnica di Filtraggio: Il Metodo di Welch

Per visualizzare l'andamento temporale della media ed eliminare il rumore, non è stata utilizzata una media cumulativa semplice, bensì una **Media Mobile a Finestra** (*Window Average*). Questo approccio è l'applicazione pratica del **Metodo di Welch** descritto nella letteratura di riferimento (Welch, 1983 - *Eq. 6.87*), definito come:

$$\tilde{\mu}(n; K) = (2K + 1)^{-1} \sum_{k=-K}^K \hat{\mu}_{n+k}$$

In OMNeT++, questo filtro è stato applicato con una finestra temporale (*Window Size*) di 5000 campioni. L'analisi grafica ha evidenziato una rapida fase di salita iniziale seguita da una stabilizzazione delle oscillazioni intorno a $t \approx 28.000$ secondi.

5.4 Configurazione Finale

Sulla base dell'analisi grafica, è stato scelto un **warmup-period** conservativo di **35.000 secondi** per garantire la completa esclusione del transiente in tutte le configurazioni. Il tempo totale di simulazione (**sim-time-limit**) è stato quindi impostato a **75.000 secondi**, garantendo una finestra di osservazione utile (*steady-state period*) di 40.000 secondi.

Di seguito si riporta la configurazione finale adottata nel file `omnetpp.ini`:

Snippet omnetpp.ini:

```

1 [General]
2 network = TwoNodeQueueing.simulations.TwoNodeNetwork
3
4 # Durata totale: 35k (warmup) + 40k (dati utili)
5 sim-time-limit = 75000s
6
7 # Numero di repliche per il metodo delle repliche
  indipendenti
8 repeat = 20
9 seed-set = ${repetition}
10
11 # Periodo di riscaldamento determinato tramite Metodo di
  Welch
12 warmup-period = 35000s

```

```

13
14 # Iterazione su tutti i parametri richiesti
15 **.source.interArrivalTime = ${4.0s, 5.0s, 6.0s}
16 **.source.userTypeProbability = ${0.4, 0.6, 0.8}
17 **.node1.feedbackProbability = ${0.6, 0.8, 0.9}

```

6 Stima della Lunghezza Media della Coda (Metrica 1)

6.1 Definizione della Metrica

La prima misura di prestazione richiesta dal progetto è la lunghezza media della coda. Questa metrica deve essere stimata per tutti i server del sistema e distinta per tipologia di utente (U_1 e U_2), oltre che calcolata in forma aggregata per il totale degli utenti.

Nel modello sviluppato, il concetto di "coda per tutti i server" si traduce nella misurazione dei seguenti punti di accumulo:

- **Nodo 1:** Sebbene servito da un unico processore, gestisce logicamente due code distinte in base alla priorità. Vengono analizzate separatamente la coda degli utenti U_1 (alta priorità) e quella degli utenti U_2 .
- **Nodo 2 (SS1):** Coda dedicata esclusivamente agli utenti U_1 in attesa del server $SS1$.
- **Nodo 2 (SS2):** Coda dedicata esclusivamente agli utenti U_2 in attesa del server $SS2$.

6.2 Procedura di Estrazione dei Dati

I dati necessari per il calcolo sono stati estratti dai file di risultato (.sca) generati da OMNeT++. Poiché l'interesse ricade sui valori medi di lungo periodo (regime stazionario), sono stati utilizzati gli scalari registrati con modalità `timeavg` nel modulo `statistic` del file NED.

Le statistiche specifiche estratte per ogni run sono:

- `queueLenNode1_U1:timeavg`: Lunghezza media temporale della coda U_1 al Nodo 1.
- `queueLenNode1_U2:timeavg`: Lunghezza media temporale della coda U_2 al Nodo 1.

- `queueLenSS1:timeavg`: Lunghezza media temporale della coda al server SS1.
- `queueLenSS2:timeavg`: Lunghezza media temporale della coda al server SS2.

I dati grezzi di tutte le 20 repliche per le 486 configurazioni simulate sono stati esportati in formato JSON tramite l'IDE di OMNeT++ per la successiva post-elaborazione.

6.3 Metodologia di Calcolo

Per ottenere la stima puntuale e gli intervalli di confidenza richiesti, è stato applicato il **Metodo delle Repliche Indipendenti**. Dato un insieme di $M = 20$ repliche indipendenti (parametro `repeat = 20`), per ogni configurazione di parametri $(m, q, p, \dots)$ e per ogni metrica specifica, si è proceduto come segue:

1. **Stima Puntuale ($\bar{X}$)**: Calcolata come la media campionaria dei valori medi ottenuti nelle singole repliche.

$$\bar{X} = \frac{1}{M} \sum_{m=1}^M X_m$$

2. **Intervallo di Confidenza (CI)**: Calcolato utilizzando la distribuzione t-Student, poiché la varianza reale della popolazione è ignota. Con $M = 20$, si utilizzano $M - 1 = 19$ gradi di libertà e un livello di confidenza del 95% ($\alpha = 0.05$).

$$CI = \bar{X} \pm t_{M-1, 1-\alpha/2} \cdot \frac{S}{\sqrt{M}}$$

dove S è la deviazione standard campionaria delle medie delle repliche.

Per il calcolo della **Coda Totale** relativa a un nodo (es. Coda Totale Nodo 1), si è proceduto sommando le medie delle singole tipologie per ogni replica *prima* dell'aggregazione statistica:

$$\text{CodaTotale}_m = \text{CodaU1}_m + \text{CodaU2}_m$$

Su questo nuovo valore aggregato sono state poi calcolate media e CI.

6.4 Implementazione del Calcolo

L'elaborazione statistica è stata automatizzata tramite uno script Python (eseguito su ambiente Google Colab) che utilizza la libreria `scipy.stats` per la determinazione precisa dei valori critici t . Di seguito si riporta un estratto significativo della logica utilizzata per il calcolo:

```
1 # Calcolo della Coda Totale per ogni singola run (replica)
2 df_pivot['CodaTotale_Nodo1'] = df_pivot['queueLenNode1_U1'] +
   df_pivot['queueLenNode1_U2']
3
4 # Raggruppamento per configurazione (escludendo la replica)
5 parametri_config = ['Config', 'm_SS1', 'm_SS2', 'm', 'q', 'p'
   ]
6 grouped = df_pivot.groupby(parametri_config)
7
8 # 1. Stima Puntuale (Media delle 20 repliche)
9 stima_puntuale = grouped.mean()
10
11 # 2. Calcolo Intervallo di Confidenza (95%)
12 n_campioni = 20
13 # Valore t-critico per 19 gradi di libert  (approx 2.093)
14 t_value = st.t.ppf(0.975, n_campioni - 1)
15 dev_std = grouped.std()
16
17 margine_errore = t_value * (dev_std / np.sqrt(n_campioni))
18
19 ci_inferiore = stima_puntuale - margine_errore
20 ci_superiore = stima_puntuale + margine_errore
```

Listing 1: Logica Python per il calcolo di Stima Puntuale e CI

7 Stima del Tempo di Soggiorno (Metrica 2)

7.1 Definizione della Metrica

La seconda misura di prestazione, fondamentale per valutare la Qualit  del Servizio (QoS) percepita dall'utente,   il **Tempo di Soggiorno** (*Sojourn Time*). Questa metrica quantifica l'intervallo temporale totale trascorso da un job all'interno della rete, calcolato come la differenza tra l'istante di uscita dal sistema (Sink) e l'istante di generazione (Source).

Il tempo di soggiorno T_s comprende:

- I tempi di attesa in tutte le code incontrate (Nodo 1 e Nodo 2).
- I tempi di servizio effettivi presso i server.

- I ritardi dovuti ai cicli di feedback (ripetizione del percorso).

L'analisi viene effettuata sia distintamente per tipologia (U_1 e U_2), per evidenziare l'impatto della priorità, sia in forma aggregata ("tutti gli utenti") per valutare la performance globale del sistema.

7.2 Procedura di Estrazione dei Dati

A differenza delle lunghezze di coda (campionate nel tempo), per il tempo di soggiorno sono stati utilizzati gli oggetti statistici aggregati (**statistic**) generati dal modulo **Sink**. Questi oggetti forniscono direttamente media, deviazione standard e numero di campioni per ogni replica.

Le statistiche estratte dai file **.anf** e convertite in JSON sono:

- **sojournTimeU1_stats**: Oggetto statistico per i job di tipo U_1 .
- **sojournTimeU2_stats**: Oggetto statistico per i job di tipo U_2 .

Ogni record estratto contiene due valori fondamentali per la post-elaborazione: la *media* (**mean**) della singola run e il *conteggio* (**count**) degli utenti serviti.

7.3 Metodologia di Calcolo

Il calcolo delle stime puntuali e degli intervalli di confidenza segue il **Metodo delle Repliche Indipendenti** già descritto per la Metrica 1. Tuttavia, il calcolo del valore aggregato "Tutti gli Utenti" richiede un approccio diverso rispetto alla semplice somma delle code.

Per ogni replica m , il tempo medio di soggiorno globale $\bar{T}_{sys,m}$ è calcolato come **media ponderata** rispetto al numero di utenti serviti per ciascuna classe:

$$\bar{T}_{sys,m} = \frac{\bar{T}_{U1,m} \cdot N_{U1,m} + \bar{T}_{U2,m} \cdot N_{U2,m}}{N_{U1,m} + N_{U2,m}}$$

Dove:

- $\bar{T}_{Ux,m}$ è il tempo medio di soggiorno per la classe x nella replica m .
- $N_{Ux,m}$ è il numero di utenti di classe x serviti nella replica m (necessario perché il numero di arrivi è stocastico e varia leggermente tra le repliche).

L'aggregazione finale di questi valori per ottenere l'Intervallo di Confidenza richiede un accorgimento fondamentale, in quanto, nell'analisi del Tempo Medio, è stata rilevata una marcata **starvation selettiva** a carico degli utenti non prioritari (U_2), causata dal meccanismo di priorità che garantisce il servizio alla classe U_1 anche in condizioni di saturazione.

Nelle repliche in cui nessun utente U_2 riesce a completare il servizio, l'assegnazione di un tempo nullo (0.0) indurrebbe una grave sottostima della media globale. Pertanto, tali occorrenze sono state trattate come **dati mancanti** (*Missing Values*) e le relative repliche escluse dal calcolo.

Di conseguenza, la statistica finale non si basa necessariamente sul numero totale di repliche simulate ($M = 20$), ma sul sottoinsieme dinamico N_{valid} di repliche che hanno prodotto un output valido. I gradi di libertà (ν) della distribuzione t -Student vengono quindi ricalcolati per ogni configurazione:

$$\nu = N_{\text{valid}} - 1 \quad (1)$$

dove $N_{\text{valid}} \leq 20$ rappresenta il numero effettivo di repliche sopravvissute alla *starvation*.

7.4 Implementazione del Calcolo

L'elaborazione è stata eseguita tramite lo script Python `TempoDiSoggiornoSimulazione2.ipynb`. Lo script gestisce l'estrazione dei campi annidati nel JSON (media e conteggio) e applica la ponderazione descritta. Di seguito lo snippet della logica implementata:

```

1 # Estrazione dei valori 'mean' e 'count' dal JSON statistico
2 # Calcolo della media pesata per il sistema globale (U1 + U2)
  per ogni Run
3
4 # Numeratore: Somma dei tempi totali (Media * Conteggio)
5 total_time_sum = (df_pivot['sojournTimeU1_stats:mean'] *
6                   df_pivot['sojournTimeU1_stats:count']) + \
7                   (df_pivot['sojournTimeU2_stats:mean'] *
8                     df_pivot['sojournTimeU2_stats:count'])
9
10 # Denominatore: Numero totale di utenti serviti
11 total_count = df_pivot['sojournTimeU1_stats:count'] +
12                df_pivot['sojournTimeU2_stats:count']
13
14 # Calcolo Tempo Medio Globale per la singola replica
15 df_pivot['SojournTime_System'] = total_time_sum / total_count
16
17 # Raggruppamento e Calcolo Statistico
18 grouped = df_pivot.groupby(parametri_config)

```



```

16
17 # 1. Stima Puntuale
18 stima_puntuale = grouped.mean()
19
20 # 2. Intervallo di Confidenza Dinamico (adatta N ai dati
    validi)
21 n_valid = grouped.count()                # Conta le repliche
    non-NaN
22 t_value = st.t.ppf(0.975, n_valid - 1) # Gradi di libertà
    dinamici
23 metric_std = grouped.std()
24
25 # Calcolo margine con N variabile
26 margine_errore = t_value * (metric_std / np.sqrt(n_valid))
27
28 ci_inferiore = stima_puntuale - margine_errore
29 ci_superiore = stima_puntuale + margine_errore

```

Listing 2: Logica Python per il calcolo del Tempo di Soggiorno Ponderato

8 Stima del tempo massimo (minimo) di permanenza nel sistema degli utenti per tipologia (Metrica 3)

8.1 Definizione della Metrica

La terza metrica richiesta riguarda la stima dei valori estremi del tempo di permanenza: il tempo massimo e il tempo minimo. In un sistema stocastico aperto, il massimo assoluto di una simulazione è una misura instabile che tende a divergere all'aumentare della durata del run (eventi rari o outlier). Pertanto, facendo riferimento alla letteratura statistica (Welch, Sez. 6.3.3.c) che suggerisce l'analisi dei quantili per caratterizzare le code della distribuzione, la stima è stata effettuata calcolando i Percentili:

- **Tempo Massimo:** Stimato tramite il **95° Percentile** ($x_{0.95}$). Indica il tempo entro il quale viene servito il 95% degli utenti, filtrando il 5% di code anomale o estreme (outlier).
- **Tempo Minimo:** Stimato tramite il **5° Percentile** ($x_{0.05}$). Rappresenta il limite inferiore realistico del tempo di attraversamento, raggiunto dal 5% degli utenti più veloci.

8.2 Procedura di Estrazione dei Dati

A differenza delle metriche precedenti basate su valori medi scalari, il calcolo dei percentili richiede l'analisi dell'intera distribuzione temporale dei tempi di soggiorno. È stato quindi necessario operare sui **Vettori** (*vector*), che registrano la storia completa di ogni singolo job generato.

La procedura seguita per l'estrazione è la seguente:

1. **Apertura File di Analisi:** Apertura del file `.anf` generato dalla simulazione in OMNeT++ e navigazione nella sezione `BrowseData` dell'interfaccia.
2. **Selezione Istogrammi/Vettori:** Accesso alla categoria `Vectors` per recuperare le serie storiche.
3. **Estrazione Statistiche ed Esportazione:** Identificazione e selezione dei vettori relativi al tempo di permanenza per tipologia:
 - `sojournTimeU1_vec`
 - `sojournTimeU2_vec`

I dati selezionati vengono come per le altre metriche esportati in formato JSON per l'elaborazione esterna.

8.3 Metodologia di Calcolo

8.4 Metodologia di Calcolo

La metodologia applicata è quella descritta in letteratura come *Metodo delle Proporzioni applicato alle Repliche Indipendenti* (cfr. Welch, Sez. 6.3.3.b).

Per la stima della **Metrica 4** (probabilità di superamento della soglia τ), l'algoritmo di calcolo prevede una gestione specifica per le repliche in stato di *starvation*. Mentre in condizioni operative normali la stima puntuale è calcolata come la frequenza relativa dei campioni che eccedono la soglia, nel caso in cui nessun utente completi il servizio ($N_{tot} = 0$) si assume che il tempo di permanenza tenda asintoticamente all'infinito. Fisicamente, ciò implica che la permanenza nel sistema supererà certamente qualsiasi soglia finita $\tau \in \{2.0, 3.0, 4.0\}$ s.

Di conseguenza, per tali repliche è stata implementata una regola di **imputazione logica** che assegna d'ufficio una probabilità di violazione pari a 1.0 (100%). Il calcolo si articola in due passi fondamentali.

8.4.1 Passo A: Calcolo della Proporzione Campionaria Intra-Replica

Per ogni singola replica m (con $m = 1 \dots 20$) e per ogni threshold T , si calcola la proporzione campionaria $\hat{p}_m$. Data la sequenza di osservazioni valide V_m nella replica m con cardinalità K_m :

1. Si conta il numero di eventi favorevoli v_m , ovvero il numero di utenti per cui il tempo di soggiorno supera la soglia:

$$v_m = \text{count}(V_m > T)$$

2. Si calcola la proporzione $\hat{p}_m$. Qui interviene la correzione per la *starvation*:

$$\hat{p}_m = \begin{cases} \frac{v_m}{K_m} & \text{se } K_m > 0 \\ 1.0 & \text{se } K_m = 0 \quad (\text{Starvation}) \end{cases} \quad (2)$$

Al termine di questa fase, si ottengono sempre **20 valori indipendenti** $\hat{p}_1, \dots, \hat{p}_{20}$ che rappresentano le stime della probabilità per ciascuna run.

8.4.2 Passo B: Statistiche Finali Inter-Replica

Poiché grazie all'imputazione non vi sono dati mancanti, i 20 valori $\hat{p}_m$ vengono trattati come un campione i.i.d. completo di dimensione $M = 20$. Su di esso si applica l'inferenza statistica standard:

1. **Stima Puntuale ($\bar{p}$)**: È la media aritmetica delle proporzioni calcolate nelle repliche:

$$\bar{p} = \frac{1}{M} \sum_{m=1}^M \hat{p}_m \quad (3)$$

2. **Intervallo di Confidenza (CI)**: Calcolato utilizzando la varianza campionaria delle proporzioni S_p^2 e la distribuzione t -Student con $M - 1 = 19$ gradi di libertà costanti:

$$CI = \bar{p} \pm t_{M-1, 1-\alpha/2} \cdot \frac{S_p}{\sqrt{M}} \quad (4)$$

8.5 Implementazione del Calcolo

Il file JSON contenente i vettori è stato processato tramite lo script Python `TempoMassimoDiPermanenzaSimulazione3.ipynb`. Lo script utilizza la libreria `numpy` per il calcolo efficiente dei percentili sui vettori numerici e `scipy.stats` per l'inferenza statistica.

Di seguito si riporta lo snippet significativo della logica utilizzata:

```

1 # 1. Iterazione su ogni singola Run (Replica)
2 for run in data:
3     # Estrazione del vettore dei tempi
4     vector_data = run['vectors']['sojournTimeU1_vec']
5
6     # Gestione Starvation: Calcolo solo se ci sono dati
7     if len(vector_data) > 0:
8         # 2. Calcolo dei Percentili sulla singola run
9         p95 = np.percentile(vector_data, 95) # Stima Max
10        p05 = np.percentile(vector_data, 5)  # Stima Min
11    else:
12        # Se il vettore è vuoto (Starvation), il dato è
13        # mancante
14        p95 = np.nan
15        p05 = np.nan
16
17    results.append({
18        'Config': run_config,
19        'Max_Estimate': p95,
20        'Min_Estimate': p05
21    })
22
23 # Creazione DataFrame e Raggruppamento
24 df = pd.DataFrame(results)
25 grouped = df.groupby(['Config', 'm', 'q', 'p'])
26
27 # 3. Calcolo Statistiche Finali (Repliche Dinamiche)
28 stima_max = grouped['Max_Estimate'].mean()
29
30 # Calcolo Intervallo di Confidenza Dinamico
31 n_valid = grouped['Max_Estimate'].count() # Conta solo i
32 # valori non-NaN
33 std_err = grouped['Max_Estimate'].sem()   # Errore Standard (
34 # std / sqrt(n))
35
36 # T-Student con gradi di libertà variabili (nu = n_valid - 1)
37 t_crit = st.t.ppf(0.975, n_valid - 1)
38
39 # Margine di errore
40 ci_margin = t_crit * std_err
41
42 ci_inferiore = stima_prob - ci_margin
43 ci_superiore = stima_prob + ci_margin

```

Listing 3: Logica Python per il calcolo dei Percentili (Max/Min)

9 Stima della percentuale di utenti, che sperimentano un tempo di permanenza nel sistema superiore al threshold 2.0, 3.0, 4.0 (Metrica 4)

9.1 Definizione della Metrica

La quarta metrica richiesta dal progetto consiste nello stimare la percentuale di utenti che sperimentano un tempo di permanenza nel sistema superiore a specifiche soglie (*threshold*). Le soglie definite dalle specifiche sono $T \in \{2.0s, 3.0s, 4.0s\}$.

Dal punto di vista statistico, questo problema equivale alla stima della probabilità p che una variabile casuale stazionaria (il tempo di soggiorno S) cada in un intervallo specifico, ovvero (T, ∞) .

$$p = P(S > T)$$

L'obiettivo è fornire una stima puntuale e un intervallo di confidenza per questa probabilità per ciascuna tipologia di utente (U_1, U_2) e per ogni configurazione.

9.2 Procedura di Estrazione dei Dati

Poiché la metrica richiede di valutare la condizione $S > T$ per ogni singolo utente servito, non è sufficiente utilizzare le medie aggregate. È necessario operare sui **Vettori** completi dei tempi di soggiorno.

La procedura di estrazione è identica a quella descritta per la Metrica 3.

9.3 Metodologia di Calcolo

La metodologia applicata è quella descritta in letteratura come **Metodo delle Proporzioni** applicato alle Repliche Indipendenti (Welch, Sez. 6.3.3.b).

Per la stima della **Metrica 4** (probabilità di superamento della soglia τ), l'algoritmo di calcolo prevede una gestione specifica per le repliche in stato di *starvation*.

Mentre in condizioni operative normali la stima puntuale è calcolata come la frequenza relativa dei campioni che eccedono la soglia ($p_m = N_{>\tau}/N_{tot}$), nel caso in cui nessun utente completi il servizio ($N_{tot} = 0$) si assume che il tempo di permanenza tenda asintoticamente all'infinito, ciò implica che la permanenza nel sistema supererà qualsiasi soglia finita $\tau \in \{2.0, 3.0, 4.0\}$ s. Di

conseguenza, per tali repliche è stata implementata una regola di imputazione logica che assegna d'ufficio una probabilità di violazione pari a **1.0** (100%). Il calcolo si articola in due passi fondamentali.

9.3.1 Passo A: Calcolo della Proporzione Campionaria Intra-Replica

Per ogni singola replica m (con $m = 1 \dots 20$) e per ogni threshold T , si calcola la proporzione campionaria $\hat{p}_m$. Data la sequenza di osservazioni valide $V_{m,n}$ nella replica m :

1. Si conta il numero di eventi favorevoli v_m , ovvero il numero di utenti per cui il tempo di soggiorno supera la soglia:

$$v_m = \text{count}(V_{m,n} > T)$$

2. Si calcola la proporzione dividendo per il numero totale di osservazioni valide N_{valid} nella replica:

$$\hat{p}_m = \frac{v_m}{N_{valid}}$$

Al termine di questa fase, si ottengono 20 valori indipendenti $\hat{p}_1, \dots, \hat{p}_{20}$ che rappresentano le stime della probabilità per ciascuna run.

9.3.2 Passo B: Statistiche Finali Inter-Replica

I 20 valori $\hat{p}_m$ vengono trattati come un campione i.i.d. su cui applicare l'inferenza statistica standard:

1. **Stima Puntuale ($\hat{p}$):** È la media aritmetica delle proporzioni calcolate nelle repliche.

$$\hat{p} = \frac{1}{M} \sum_{m=1}^M \hat{p}_m$$

2. **Intervallo di Confidenza (CI):** Calcolato utilizzando la varianza campionaria delle proporzioni $s^2(\hat{p}_m)$ e la distribuzione t-Student con $M - 1$ gradi di libertà:

$$CI = \hat{p} \pm t_{M-1, 1-\alpha/2} \cdot \frac{s(\hat{p}_m)}{\sqrt{M}}$$

9.4 Implementazione del Calcolo

Il calcolo è stato automatizzato tramite lo script Python `Stima della Percentuale di Utenti Simulazione4.ipynb`. Lo script itera su tutte le configurazioni, applica il filtro vettoriale per ogni soglia e aggrega i risultati.

Di seguito lo snippet significativo della logica utilizzata:

```
1 # 1. Iterazione su ogni singola Run (Replica)
2 for run in data:
3     # Estrazione del vettore dei tempi
4     vector_data = run['vectors']['sojournTimeU2_vec']
5
6     # --- Passo A: Calcolo Proporzione Intra-Replica ---
7     # Implementazione logica per la gestione della Starvation
8     if len(vector_data) == 0:
9         # Se il vettore è vuoto (Starvation), il tempo tende
10        a infinito.
11        # Quindi supera sicuramente la soglia: Probabilità =
12        1.0 (100%)
13        p_hat = 1.0
14    else:
15        # Calcolo standard: Frequenza relativa (utenti >
16        soglia / totale)
17        count_over_threshold = np.sum(vector_data > THRESHOLD
18        )
19        p_hat = count_over_threshold / len(vector_data)
20
21    results.append({
22        'Config': run_config,
23        'Prob_Estimate': p_hat
24    })
25
26 # Creazione DataFrame e Raggruppamento
27 df = pd.DataFrame(results)
28 grouped = df.groupby(['Config', 'm', 'q', 'p'])
29
30 # --- Passo B: Statistiche Finali (Inter-Replica) ---
31 # Calcolo Stima Puntuale (Media delle probabilità)
32 stima_prob = grouped['Prob_Estimate'].mean()
33
34 # Calcolo Intervallo di Confidenza
35 n_campioni = grouped['Prob_Estimate'].count()
36 std_err = grouped['Prob_Estimate'].sem() # Deviazione std /
37 sqrt(n)
38
39 # T-Student (df = n_campioni - 1)
40 t_crit = st.t.ppf(0.975, n_campioni - 1)
```

```

37 # Margine di errore
38 ci_margin = t_crit * std_err
39
40 ci_inferiore = stima_prob - ci_margin
41 ci_superiore = stima_prob + ci_margin

```

Listing 4: Logica Python per il Metodo delle Proporzioni

10 Analisi dei risultati

L'analisi incrociata delle quattro metriche di prestazione, riassunta nelle Tabelle 3, 4 e 5, offre un quadro inequivocabile sui limiti operativi del sistema. I dati confermano che le prestazioni della rete sono governate da tre fattori critici: l'efficienza del nodo 1 (Configurazione), la probabilità di feedback (p) e la composizione dell'utenza (q).

Di seguito si discute il comportamento del sistema in relazione a questi fattori.

10.1 Analisi di Stabilità e Impatto del Feedback

La Tabella 3 evidenzia la fragilità del sistema rispetto alla probabilità di feedback p . Si osserva un passaggio non lineare, bensì critico, da una condizione di stabilità a una di saturazione completa.

- **Stabilità ($p = 0.6$):** Con un feedback moderato, la Configurazione 1 gestisce il traffico in modo efficiente. Il tempo medio di soggiorno per gli utenti U2 è di circa 171 secondi, con una coda totale gestibile (≈ 16 utenti).
- **Il Collasso ($p \geq 0.8$):** Superata la soglia di $p = 0.8$, il sistema collassa. Il tempo medio per U2 esplode a oltre 29.000 secondi.
- **Il Collo di Bottiglia Nascosto:** I dati mostrano che la saturazione non è confinata al Nodo 1. Con $p = 0.9$, la coda totale del sistema (6571 utenti) supera significativamente la coda del solo Nodo 1 (≈ 5080 utenti), indicando che circa 1500 utenti sono in coda al Nodo 2. Il feedback intasa globalmente la rete, rendendo inefficace la velocità del primo servente.

Tabella 3: Impatto del Feedback (p) sulla Configurazione 1. Si nota il passaggio critico da sistema stabile ($p = 0.6$) a sistema saturo ($p \geq 0.8$). (Scenario: $m = 6.0, q = 0.4$)

Scenario	Metrica 1 (Code)		Metrica 2 (T. Medio)		Metrica 3 (95%ile)		Metrica 4
Parametri (p)	N1_U2	Totale Sys	T_U1 (s)	T_U2 (s)	Max U1	Max U2	P(U2 > 4s)
$p = 0.6$ (Stabile)	16.03	16.55	16.10	171.76	50.70	581.33	0.97
$p = 0.8$ (Saturo)	4026.02	4030.22	84.23	29359.62	289.52	53652.26	1.00
$p = 0.9$ (Critico)	5080.63	6571.70	15113.06	36452.24	40900.72	58839.70	1.00

10.2 Efficacia della Priorità (Scenario Stabile)

La Tabella 4 analizza l'effetto della priorità in condizioni operative normali ($p = 0.6$), permettendo di valutare la reale differenziazione del servizio.

- **Differenziazione:** Con $q = 0.4$, la priorità garantisce agli utenti U1 un vantaggio enorme. Il tempo medio di soggiorno è di 16.1s contro i 171.8s degli utenti U2. Gli utenti U1 attraversano il sistema circa 10 volte più velocemente, confermando che la disciplina al Nodo 1 funziona efficacemente proteggendo gli utenti U1 dal traffico ordinario.
- **Concorrenza Interna (Effetto su U1):** All'aumentare della percentuale degli utenti U1 ($q = 0.8$), si nota un leggero degrado delle prestazioni per gli utenti U1 stessi (da 16.1s a 23.9s). Questo accade perché, essendoci più utenti prioritari, essi iniziano a formare code "interne" tra di loro al primo nodo, riducendo il vantaggio relativo.
- **Paradosso del Miglioramento U2:** Contrariamente a quanto accade nel sistema saturo, qui all'aumentare di q i tempi medi degli utenti U2 migliorano (scendono da 171s a 96s).
 - **Spiegazione:** Sebbene gli U2 abbiano priorità bassa, un q alto significa che ci sono pochi utenti U2 che entrano nel sistema. Inoltre, gli utenti U1 hanno un tempo di servizio medio inferiore ($[1.4, 2.6] \approx 2.0s$) rispetto agli U2 ($[1.0, 4.0] \approx 2.5s$). Quindi, aumentando q , il carico totale di lavoro medio che arriva al server diminuisce, liberando risorse anche per i pochi utenti U2 presenti.

Tabella 4: Efficacia della disciplina a priorità in condizioni di stabilità. Si nota come il sistema garantisca tempi eccellenti agli utenti U1 rispetto agli U2. All'aumentare di q , gli utenti U1 iniziano a competere tra loro (i tempi salgono leggermente), mentre il carico totale sugli U2 diminuisce. (Scenario: Config 1, $m = 6.0, p = 0.6$)

Scenario	Metrica 1 (Code)		Metrica 2 (T. Medio)		Metrica 3 (95%ile)		Metrica 4
Parametri (q)	N1_U1	N1_U2	T_U1 (s)	T_U2 (s)	Max U1	Max U2	P(U2 > 4s)
$q = 0.4$	0.32	16.03	16.10	171.76	50.70	581.33	0.97
$q = 0.6$	0.60	6.39	18.62	106.12	59.68	383.46	0.95
$q = 0.8$	1.15	2.92	23.87	96.50	79.45	361.41	0.92

10.3 Confronto Strutturale (Config 1 vs Config 2)

La Tabella 5 dimostra l'inadeguatezza strutturale della Configurazione 2.

- **Saturazione Immediata:** A parità di carico ridotto ($p = 0.6, m = 6.0$), dove la Configurazione 1 è stabile ($T_{U2} \approx 171s$), la Configurazione 2 è già in profonda saturazione ($T_{U2} \approx 15.917s$).
- **Code:** La coda totale passa da 16 utenti (Config 1) a oltre 1871 utenti (Config 2). Questo conferma che i tempi di servizio della Configurazione 2 (distribuzione uniforme con media più alta) sono matematicamente insufficienti a smaltire anche il carico di base, rendendo questa configurazione non operativa a prescindere dal feedback.

10.4 Analisi e Coerenza con le Specifiche

L'analisi delle metriche 3-4 conferma la criticità del sistema:

- **Violazione SLA:** La probabilità che un utente U2 superi i 4 secondi di permanenza ($P(T > 4s)$) è pari a 1.0 (100%) in quasi tutti gli scenari instabili. Il sistema non è in grado di garantire i livelli di servizio richiesti.
- **Coerenza con il Progetto (var-8.pdf):** I risultati ottenuti sono pienamente coerenti con le specifiche della variante assegnata.
 - Il "fattore moltiplicativo" del traffico dato dal feedback ($p = 0.9 \rightarrow 10$ visite medie) genera un carico $\lambda_{eff} \gg \mu$, spiegando matematicamente i valori "catastrofici" osservati.

- La disparità tra T_{U1} e T_{U2} conferma la corretta implementazione della priorità (differente dal *Processor Sharing* del modello teorico in `progetto-8.pdf`).
- La presenza di code significative anche al di fuori del Nodo 1 valida la complessità introdotta dalla topologia multi-servente al Nodo 2.

In conclusione, l'unica configurazione sostenibile risulta essere la Configurazione 1 con un feedback strettamente limitato ($p \leq 0.6$). Qualsiasi altro scenario porta inevitabilmente alla saturazione della rete. Il modello analizzato è pienamente conforme alla definizione di variante descritta in 'var-8.pdf', e si distingue dal modello teorico di riferimento ('progetto-8.pdf') principalmente per l'introduzione della disciplina a priorità al Nodo 1 e per la topologia multi-servente al Nodo 2. I risultati ottenuti sono coerenti con le aspettative teoriche: i parametri di feedback assegnati ($p \geq 0.8$) generano un carico di feedback tale da violare sistematicamente la condizione di stabilità ($\rho > 1$) per le configurazioni di servizio date, portando alla saturazione osservata nelle metriche.

Tabella 5: Confronto strutturale tra Configurazione 1 (servente Veloce) e Configurazione 2 (servente Lento) a basso carico. La Config 2 mostra segni di saturazione immediata. (Scenario: $m = 6.0, p = 0.6, q = 0.4$)

Scenario	Metrica 1 (Code)		Metrica 2 (T. Medio)		Metrica 3 (95%ile)		Metrica 4
Configurazione	N1_U2	Totale Sys	T_U1 (s)	T_U2 (s)	Max U1	Max U2	P(U2 > 4s)
Config 1 (Ottimale)	16.03	16.55	16.10	171.76	50.70	581.33	0.97
Config 2 (Critica)	1870.56	1871.30	22.58	15917.53	68.65	34401.86	1.00

A Appendice: Tabelle dei Risultati

In questa sezione sono riportati i dati completi delle simulazioni.

Configurazione Generale dei parametri:

- Tempo Medio Interarrivi (m): 6.0
- Tempo Medio Servizio SS1 (m_{SS1}): 3.0
- Tempo Medio Servizio SS2 (m_{SS2}): 2.0
- Soglia Threshold: 4.0

Tabella 6: Metrica 1: Lunghezza media delle code (Dettaglio per Nodo e Utente).

Config	p	q	N1_U1	N1_U2	N2_SS1	N2_SS2	TOT	CI 95% (N1)
Config 1	0.6	0.4	0.32	16.03	0.12	0.08	16.55	[11.8-20.9]
	0.8	0.4	1.24	4026.02	2.90	0.06	4030.22	[3985.1-4069.4]
	0.9	0.4	2.20	5080.63	1488.84	0.03	6571.70	[5050.5-5115.1]
	0.6	0.6	0.60	6.39	0.32	0.04	7.34	[6.4-7.5]
	0.8	0.6	3.52	3061.57	725.72	0.01	3790.82	[3041.3-3088.9]
	0.9	0.6	3.49	3383.06	3137.33	0.02	6523.90	[3357.5-3415.7]
	0.6	0.8	1.15	2.92	0.76	0.01	4.85	[3.7-4.5]
	0.8	0.8	7.62	1555.18	2221.51	0.01	3784.32	[1540.5-1585.1]
	0.9	0.8	7.46	1692.40	4803.76	0.01	6503.62	[1676.5-1723.2]
Config 2	0.6	0.4	0.60	1870.56	0.09	0.03	1871.30	[1812.7-1929.6]
	0.8	0.4	70.74	5442.19	1.56	0.00	5514.49	[5476.2-5549.7]
	0.9	0.4	1847.75	5496.90	4.21	0.00	7348.87	[7300.0-7389.3]
	0.6	0.6	2.07	1847.21	0.25	0.01	1849.55	[1804.8-1893.8]
	0.8	0.6	1867.65	3672.78	1.69	0.00	5542.13	[5496.5-5584.4]
	0.9	0.6	3681.71	3676.36	3.99	0.00	7362.06	[7313.3-7402.8]
	0.6	0.8	99.03	1758.61	0.49	0.00	1858.13	[1805.4-1909.9]
	0.8	0.8	3690.11	1825.39	1.67	0.00	5517.17	[5473.4-5557.6]
	0.9	0.8	5503.82	1840.96	4.14	0.00	7348.91	[7302.4-7387.2]

Tabella 7: Metrica 2: Tempi Medi e Statistiche SLA.

Config	p	q	Tempo U1 (s)	Tempo U2 (s)	CI 95% (U2)
Config 1	0.6	0.4	16.10	171.76	[126.39 - 217.14]
	0.8	0.4	84.23	29359.62	[29072.98 - 29646.26]
	0.9	0.4	15113.06	36452.24	[36048.15 - 36856.32]
	0.6	0.6	18.62	106.12	[98.22 - 114.02]
	0.8	0.6	6195.72	34782.74	[34414.55 - 35150.93]
	0.9	0.6	18075.07	36181.96	[35665.37 - 36698.55]
	0.6	0.8	23.87	96.50	[85.66 - 107.34]
	0.8	0.8	11657.75	35348.42	[34689.39 - 36007.44]
	0.9	0.8	18807.98	36629.04	[35747.84 - 37510.24]
Config 2	0.6	0.4	22.58	15917.53	[15462.57 - 16372.49]
	0.8	0.4	1096.88	54849.23	[46022.04 - 63676.42]
	0.9	0.4	19719.80	—	—
	0.6	0.6	35.35	22717.59	[22210.71 - 23224.46]
	0.8	0.6	14933.60	—	—
	0.9	0.6	24638.27	—	—
	0.6	0.8	751.04	50365.98	[40714.90 - 60017.06]
	0.8	0.8	20746.84	—	—
	0.9	0.8	27095.80	—	—

Tabella 8: Metrica 3: Percentili dei Tempi di Soggiorno (Minimo 5% e Massimo 95%).

Config	p	q	Percentili U1		Percentili U2		
			Min (5%)	Max (95%)	Min (5%)	Max (95%)	CI 95% (Max U2)
Config 1	0.6	0.4	2.31	50.70	8.59	581.33	[433.1 - 729.6]
	0.8	0.4	3.55	289.52	14110.72	53652.26	[52796.0 - 54508.5]
	0.9	0.4	4.61	40900.72	20946.49	58839.70	[57713.9 - 59965.5]
	0.6	0.6	2.28	59.68	3.99	383.46	[350.1 - 416.8]
	0.8	0.6	4.62	19410.51	19985.01	56521.98	[55840.1 - 57203.8]
	0.9	0.6	4.67	48093.01	20682.69	58019.22	[56815.7 - 59222.7]
	0.6	0.8	2.25	79.45	3.20	361.41	[309.5 - 413.3]
	0.8	0.8	5.65	34746.17	21135.18	56651.04	[55623.5 - 57678.6]
	0.9	0.8	5.75	50283.42	22244.49	57857.31	[55805.4 - 59909.3]
Config 2	0.6	0.4	3.73	68.65	6604.95	34401.86	[33424.4 - 35379.3]
	0.8	0.4	148.81	3261.83	48864.19	61385.90	[53598.8 - 69173.0]
	0.9	0.4	4282.25	44619.97	—	—	—
	0.6	0.6	4.35	114.23	11242.57	43480.19	[42901.9 - 44058.5]
	0.8	0.6	3947.30	35719.70	—	—	—
	0.9	0.6	7141.14	51370.16	—	—	—
	0.6	0.8	192.13	2032.16	46247.62	57024.12	[48746.9 - 65301.3]
	0.8	0.8	6916.05	44732.04	—	—	—
	0.9	0.8	9588.24	53992.97	—	—	—

Tabella 9: Metrica 4: Probabilità di superamento della soglia SLA ($T > 4.0s$). Valori prossimi a 1.0000 indicano violazione sistematica.

Config	p	q	Prob U1 ($> 4s$)	Prob U2 ($> 4s$)	CI 95% (U2)
Config 1	0.6	0.4	0.7630	0.9704	[0.964 - 0.976]
	0.8	0.4	0.9319	1.0000	[1.000 - 1.000]
	0.9	0.4	0.9610	1.0000	[1.000 - 1.000]
	0.6	0.6	0.7889	0.9464	[0.942 - 0.951]
	0.8	0.6	0.9624	1.0000	[1.000 - 1.000]
	0.9	0.6	0.9637	1.0000	[1.000 - 1.000]
	0.6	0.8	0.8231	0.9214	[0.913 - 0.929]
	0.8	0.8	0.9753	1.0000	[1.000 - 1.000]
	0.9	0.8	0.9769	1.0000	[1.000 - 1.000]
Config 2	0.6	0.4	0.9291	1.0000	[1.000 - 1.000]
	0.8	0.4	0.9992	1.0000	[1.000 - 1.000]
	0.9	0.4	1.0000	1.0000	[1.000 - 1.000]
	0.6	0.6	0.9642	1.0000	[1.000 - 1.000]
	0.8	0.6	1.0000	1.0000	[1.000 - 1.000]
	0.9	0.6	1.0000	1.0000	[1.000 - 1.000]
	0.6	0.8	0.9988	1.0000	[1.000 - 1.000]
	0.8	0.8	1.0000	1.0000	[1.000 - 1.000]
	0.9	0.8	1.0000	1.0000	[1.000 - 1.000]

B Analisi Dettagliata della Stabilità e dell'Errore Relativo

In questa sezione viene presentata un'analisi approfondita della stabilità statistica delle simulazioni per la Metrica 1 (Lunghezza Code), introducendo il calcolo dell'Errore Relativo Percentuale ($Err\%$). Questo indicatore è fondamentale per interpretare correttamente l'affidabilità dei dati in scenari a forte varianza o saturazione.

B.1 Definizione degli Indicatori di Qualità

L'Errore Relativo Percentuale è definito come il rapporto tra la semi-ampiezza dell'intervallo di confidenza al 95% e il valore medio stimato:

$$Err\% = \frac{\text{Ampiezza CI}}{2 \times \text{Media}} \times 100$$

Sulla base di questo valore, è stata assegnata un'etichetta di **Qualità** alla stima:

- **OTTIMO** ($Err\% < 5\%$): Indica una stima estremamente precisa con bassa varianza relativa.

- **BUONO** ($5\% \leq Err\% < 10\%$): Indica una stima affidabile.
- **INSTABILE / CRITICO** ($Err\% \geq 10\%$): Indica che il sistema è soggetto a forte varianza stocastica o che la simulazione non ha raggiunto una convergenza stabile (tipico di code molto corte o fasi transitorie).

Tabella 10: Analisi di Stabilità e Saturazione (Metrica 1 Aggregata). Scenario: $m = 6.0$, $SS1 = 3.0$, $SS2 = 2.0$.

Scenario	Config	N1_U1	N1_U2	N2_Tot	Tot_Sys	Err %	Qualità	Stato Sistema
$p = 0.6, q = 0.4$	Config 1	0.32	16.03	0.20	16.5	28.0%	CRITICO	Stabile
$p = 0.8, q = 0.4$	Config 1	1.24	4026.02	2.96	4030.2	1.0%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.4$	Config 1	2.20	5080.63	1488.87	6571.7	0.6%	Ottimo	SATURAZIONE
$p = 0.6, q = 0.6$	Config 1	0.60	6.39	0.36	7.3	7.9%	Buono	Stabile
$p = 0.8, q = 0.6$	Config 1	3.52	3061.57	725.73	3790.8	0.8%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.6$	Config 1	3.49	3383.06	3137.35	6523.9	0.9%	Ottimo	SATURAZIONE
$p = 0.6, q = 0.8$	Config 1	1.15	2.92	0.77	4.8	10.1%	Instabile	Stabile
$p = 0.8, q = 0.8$	Config 1	7.62	1555.18	2221.52	3784.3	1.4%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.8$	Config 1	7.46	1692.40	4803.77	6503.6	1.4%	Ottimo	SATURAZIONE
$p = 0.6, q = 0.4$	Config 2	0.60	1870.56	0.13	1871.3	3.1%	Ottimo	SATURAZIONE
$p = 0.8, q = 0.4$	Config 2	70.74	5442.19	1.56	5514.5	0.7%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.4$	Config 2	1847.75	5496.90	4.21	7348.9	0.6%	Ottimo	SATURAZIONE
$p = 0.6, q = 0.6$	Config 2	2.07	1847.21	0.26	1849.5	2.4%	Ottimo	SATURAZIONE
$p = 0.8, q = 0.6$	Config 2	1867.65	3672.78	1.69	5542.1	0.8%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.6$	Config 2	3681.71	3676.36	3.99	7362.1	0.6%	Ottimo	SATURAZIONE
$p = 0.6, q = 0.8$	Config 2	99.03	1758.61	0.49	1858.1	2.8%	Ottimo	SATURAZIONE
$p = 0.8, q = 0.8$	Config 2	3690.11	1825.39	1.67	5517.2	0.8%	Ottimo	SATURAZIONE
$p = 0.9, q = 0.8$	Config 2	5503.82	1840.96	4.14	7348.9	0.6%	Ottimo	SATURAZIONE

B.2 Note Interpretative dei Risultati

L'analisi della Tabella 10 rivela un apparente paradosso statistico che necessita di corretta interpretazione:

1. **Il Paradosso della Qualità Statistica:** Si osserva che le configurazioni stabili (es. Config 1, $p = 0.6$) presentano spesso una qualità classificata come "CRITICO" o "Instabile" ($Err\% > 10\%$), mentre le configurazioni sature mostrano una qualità "Ottimo" ($Err\% < 1\%$).

- *Spiegazione:* In un sistema stabile con code molto corte (es. media 16 utenti), la varianza naturale del processo di arrivo e servizio ha un peso percentuale elevato rispetto alla media. Questo "rumore" non indica un malfunzionamento, ma la natura stocastica del sistema a basso carico.

- *Contrario*: In saturazione, la coda cresce linearmente e costantemente verso valori enormi (es. 5000+). La componente di varianza diventa trascurabile rispetto alla grandezza dell'accumulo, rendendo la stima dell'errore relativo piccolissima. Pertanto, una qualità "Ottimo" su una coda di 5000 utenti certifica matematicamente lo stato di saturazione irreversibile.

2. **Collasso Strutturale della Configurazione 2**: La tabella conferma l'inadeguatezza della Configurazione 2. Anche nello scenario di minimo carico ($p = 0.6, q = 0.4$), dove la Config 1 mantiene una coda di ≈ 16 utenti, la Config 2 presenta già un accumulo di ≈ 1871 utenti. L'errore relativo basso (3.1%) conferma che questo non è un transiente, ma uno stato stazionario di saturazione.
3. **Analisi del Collo di Bottiglia (Config 1, $p = 0.9$)**: Osservando la disaggregazione delle code per lo scenario critico ($p = 0.9$), emerge che la saturazione non è limitata al Nodo 1.

- Coda Nodo 1 (U2): ≈ 5080 utenti.
- Coda Nodo 2 (Totale): ≈ 1488 utenti.

Questo dato dimostra che, con un feedback elevato ($p = 0.9$), il sistema soffre di congestione globale: il ricircolo degli utenti sovraccarica anche il secondo stadio di servizio, amplificando i tempi di soggiorno complessivi.

B.3 Analisi Approfondita dei Tempi di Soggiorno

I risultati esposti nella Tabella 11 completano il quadro prestazionale, mostrando l'impatto della saturazione sulla qualità dell'esperienza utente (QoE) in termini di ritardo temporale.

Tabella 11: Analisi dei Tempi di Soggiorno (Metrica 2 Aggregata). Scenario: $m = 6.0$, $SS1 = 3.0$, $SS2 = 2.0$.

Scenario	Config	Tempo U1 (s)	Tempo U2 (s)	Err U2 %	Qualità	Note
$p = 0.6, q = 0.4$	Config 1	16.1	171.8	26.4%	CRITICO	Disparità Estrema
$p = 0.8, q = 0.4$	Config 1	84.2	29359.6	1.0%	Ottimo	SATURAZIONE (>10ks)
$p = 0.9, q = 0.4$	Config 1	15113.1	36452.2	1.1%	Ottimo	SATURAZIONE (>10ks)
$p = 0.6, q = 0.6$	Config 1	18.6	106.1	7.4%	Buono	Stabile
$p = 0.8, q = 0.6$	Config 1	6195.7	34782.7	1.1%	Ottimo	SATURAZIONE (>10ks)
$p = 0.9, q = 0.6$	Config 1	18075.1	36182.0	1.4%	Ottimo	SATURAZIONE (>10ks)
$p = 0.6, q = 0.8$	Config 1	23.9	96.5	11.2%	Instabile	Stabile
$p = 0.8, q = 0.8$	Config 1	11657.8	35348.4	1.9%	Ottimo	SATURAZIONE (>10ks)
$p = 0.9, q = 0.8$	Config 1	18808.0	36629.0	2.4%	Ottimo	SATURAZIONE (>10ks)
$p = 0.6, q = 0.4$	Config 2	22.6	15917.5	2.9%	Ottimo	SATURAZIONE (>10ks)
$p = 0.8, q = 0.4$	Config 2	1096.9	54849.2	16.1%	Instabile	SATURAZIONE (>10ks)
$p = 0.9, q = 0.4$	Config 2	19719.8	—	—	N/A	STARVATION (NaN)
$p = 0.6, q = 0.6$	Config 2	35.3	22717.6	2.2%	Ottimo	SATURAZIONE (>10ks)
$p = 0.8, q = 0.6$	Config 2	14933.6	—	—	N/A	STARVATION (NaN)
$p = 0.9, q = 0.6$	Config 2	24638.3	—	—	N/A	STARVATION (NaN)
$p = 0.6, q = 0.8$	Config 2	751.0	50366.0	19.2%	Instabile	SATURAZIONE (>10ks)
$p = 0.8, q = 0.8$	Config 2	20746.8	—	—	N/A	STARVATION (NaN)
$p = 0.9, q = 0.8$	Config 2	27095.8	—	—	N/A	STARVATION (NaN)

I risultati esposti nella Tabella 11 completano il quadro prestazionale, mostrando l'impatto della saturazione sulla qualità dell'esperienza utente (QoE) in termini di ritardo temporale.

1. **Il Punto di Rottura della Configurazione 1:** Il passaggio da uno stato di stabilità ($p = 0.6$) a uno di saturazione ($p = 0.8$) è caratterizzato da una discontinuità "a muro verticale".

- A $p = 0.6$, il tempo medio per U2 è di 171.8s.
- A $p = 0.8$, il tempo esplode a 29359.6s.

Questo conferma che il sistema non degrada gradualmente, ma collassa non appena il fattore di ricircolo ($1/(1 - p)$) spinge il traffico effettivo oltre la soglia di stabilità $\rho = 1$.

2. **La Trappola per gli Utenti Prioritari (U1):** È fondamentale notare che la saturazione annulla parzialmente il beneficio della priorità.

Nello scenario critico ($p = 0.9, q = 0.4$), anche gli utenti U1, teoricamente privilegiati, subiscono un tempo medio di 15113s (oltre 4 ore).

- *Causa:* Sebbene saltino la coda al Nodo 1, gli utenti U1 rimangono bloccati nelle code chilometriche del Nodo 2 e nel loop di feedback, subendo il ritardo sistemico globale.

3. **Il Fenomeno della Starvation Totale (NaN):** Per la Configurazione 2 con carichi elevati ($p \geq 0.8$), i valori per gli utenti U2 non sono calcolabili (NaN).

- *Significato Fisico:* Questo indica uno stato di *starvation* completa. La coda cresce così rapidamente che, durante l'intero orizzonte temporale della simulazione, quasi nessun utente U2 riesce a completare il servizio e uscire dal sistema. Il tempo di soggiorno tende matematicamente all'infinito.

4. **L'Effetto "Crowding Out" della Priorità:** Analizzando la Configurazione 2 in saturazione, si osserva che l'aumento della percentuale di utenti U1 (q) peggiora drasticamente la situazione per gli utenti U2.

- Con $q = 0.4$ (pochi U1), il tempo U2 è $\approx 15917s$.
- Con $q = 0.8$ (molti U1), il tempo U2 sale a $\approx 50366s$.

In un sistema saturo, il server è perennemente occupato a servire la classe prioritaria, relegando la classe U2 ai soli "interstizi" di tempo libero, che diventano sempre più rari all'aumentare di q .